

Entity Query Language

Syntax, Semantics, and Complexity

Anonymous Author(s)

October 6, 2026

Contents

1	Overview	1
1.1	Motivation and Design Principles	2
1.2	Relation to Existing Query Languages	2
2	Vocabulary and the Knowledge Base	3
2.1	Signature	3
2.2	The Knowledge Base	4
2.3	Variable Symbols	4
3	Syntax	5
3.1	Terms	5
3.2	Atomic Formulas	6
3.3	Formulas (Query Bodies)	6
3.4	Queries	6
3.5	Result Quantifiers	7
3.6	Rules	7
4	Illustrative Example	7
5	Semantics	9
5.1	Domain Closure and Unique Names	9
5.2	Evaluation Semantics	9
5.3	Answer Set	10
5.4	Aggregation Semantics	10
5.5	Negation as Failure	10
5.6	Semantics of Terms	10
5.7	Semantics of Atoms	11
6	Computational Complexity	11

1 Overview

Entity Query Language (EQL) is a symbolic, declarative, relational query language with a First Order Logic like syntax that is embedded natively in Python, designed to make Python object-oriented domain models directly queryable without any additional representation layer. It separates the symbolic representation of a query from its evaluation, so that the same query can be evaluated by different backends: in memory, translated to SQL over a relational database, or by a probabilistic backend.

This document specifies the default backend, which evaluates queries in memory under closed-world reasoning: domain closure and negation as failure.

This document provides a formal specification of EQL’s syntax, semantics, and computational complexity.

1.1 Motivation and Design Principles

Existing query languages impose a representational gap between the domain model and the query layer. SQL operates over flat relational tables and requires either manual schema maintenance or an Object-Relational Mapping (ORM) layer to bridge the gap to object-oriented code. SPARQL operates over RDF triples, requiring a separate knowledge graph representation. Datalog and its extensions operate over symbolic facts that must be populated from the application separately. In all cases, the developer must maintain two representations of the same domain: one in their programming language, one in the query system.

EQL eliminates this gap. Its design rests on the following principles:

1. **No impedance mismatch.** EQL operates directly on live Python objects. There is no schema to define, no tables to populate, and no ORM to configure. The object-oriented domain model the developer already writes *is* the knowledge base.
2. **Executable queries as first-class Python.** Queries are Python expressions, not strings passed to an external compiler. This means queries are statically analysable by development tools, debuggable at runtime, and subject to normal Python error handling and type checking. There is no compilation boundary between query and application code.
3. **General programs as query components.** Any Python callable—including arbitrary programs—can participate in a query as a predicate or symbolic function, subject to determinism and purity requirements. This goes beyond the built-in predicates of SQL or the fixed vocabulary of SPARQL.
4. **Structural depth.** EQL queries traverse nested objects, lists, and dictionaries natively. There is no restriction to flat relational structure. An attribute of a field that is itself a list of objects is queried by index or method access, not by a join.
5. **Axioms in class definitions.** A class can state its axioms as class methods that return an EQL condition characterizing its members. The axioms live in the class definition alongside the data model, can be inspected, and can be used as constraints or as the bodies of rules.

1.2 Relation to Existing Query Languages

EQL’s logical foundations are drawn from conjunctive query evaluation over finite relational structures [3, 2], Negation as Failure [4], and the TBox/ABox distinction of Description Logics [1]. It differs from these traditions in three key respects.

SQL and relational databases: SQL requires a separate schema, table-based storage, and an ORM layer to map objects to relations. EQL queries the object model directly; attribute chains replace most joins, and the remaining ones are expressed by shared variables and membership atoms.

Datalog: Datalog rules are stored in the knowledge base and applied to a fixpoint. In EQL, rules are user-invoked programs; there is no fixpoint. EQL’s fact base is always a flat finite set of ground object instances. Even though rules exist they are not automatically invoked when answering queries unless they are called as part of the query.

SPARQL and RDF: SPARQL requires a triple-store representation distinct from the application model. EQL operates on the application’s own Python objects with no translation step.

Contributions. This document makes the following formal contributions:

1. A typed signature $\Sigma = (\mathcal{S}, \mathcal{C}, \mathcal{M}, \mathcal{P}, \mathcal{F})$ for EQL with a formal TBox/ABox knowledge base structure and a subsumption ordering on class symbols.
2. A distinction between query variables $\mathcal{V}_Q$ (a class annotation and a finite domain, given or taken from the objects in memory) and inference variables $\mathcal{V}_I$ (class-annotated, domain populated from rule body answer sets), formalising EQL’s rule mechanism.
3. An abstract syntax covering terms with structural depth (field, index, and method access), three atom forms including Boolean term atoms $[t]_{\mathbb{B}}$, body formulas, conjunctive queries, unions, result quantifiers, and rules.
4. A model-theoretic semantics in the single closed-world interpretation of the knowledge base (Reiter’s closed-world assumption), with negation as failure, and the γ operator of relational algebra for aggregation.
5. Computational complexity bounds for progressively richer EQL query classes, distinguishing data complexity from combined complexity.

2 Vocabulary and the Knowledge Base

2.1 Signature

A *signature* $\Sigma = (\mathcal{S}, \mathcal{C}, \mathcal{M}, \mathcal{P}, \mathcal{F})$ consists of non-logical symbols over which the EQL language is built. Variable symbols $\mathcal{V}$ are logical symbols introduced separately in Section 2.3.

Class symbols. A finite set of *class symbols* $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ corresponds to Python classes. The class hierarchy defined by the developer induces a *subsumption ordering* $\sqsubseteq$ on $\mathcal{S}$, where $S' \sqsubseteq S$ if and only if S' is a subclass of S in Python’s class system (i.e., `issubclass(S', S)` holds).

Constant symbols. A finite set of *constant symbols* $\mathcal{C}$ denotes the Python object instances populating the domain. Each constant is typed by one or more classes via the class hierarchy.

Mapped symbols. A finite set of *mapped symbols* $\mathcal{M}$. Each $m \in \mathcal{M}$ is a named attribute of a class $S \in \mathcal{S}$, representing a property that is intrinsic to S and accessible on any instance of S via dot notation. They are referred to as Mapped symbols because they can be seen as a mapping applied to S . The attribute m may be:

- a *field*, typed $m : S \rightarrow T$, returning a value of type T .
- an *indexable field*, typed $m : S \rightarrow \text{Indexable}[K, T]$, supporting element access by key $k \in K$, where K is any hashable key type (e.g., $\mathbb{Z}$ (i.e., integers) for lists and tuples, or an arbitrary hashable type for dictionaries).
- a *method*, typed $m : S \times T_1 \times \dots \times T_n \rightarrow T$, invocable with arguments.

How these attributes are accessed syntactically is defined in the term grammar (Section 3).

Predicate symbols. A finite set of *predicate class symbols* $\mathcal{P}$. Each $p \in \mathcal{P}$ is a Python class that is a subclass of `Predicate` and implements a characteristic function via `__call__(self) -> bool`. Predicates occupy two roles:

1. *Constraint role* (query body): the expression $p(\bar{t})$ is an atomic formula whose truth value is determined at evaluation time by Boolean-casting the ground predicate instance (Section 5.7).
2. *Domain role* (fact base): instances of p are ordinary Python objects and may be stored in $\mathcal{A}$, queried over, and used as arguments in other atoms.

Function symbols. A finite set of *symbolic functions* $\mathcal{F}$. Each $f \in \mathcal{F}$ is a Python callable of one of the following forms:

1. A Python function decorated with `@symbolic_function`.
2. A class method accessed through an EQL variable.
3. A subclass of `SymbolicFunction`, the class form of a symbolic function.

Evaluation stores no function outputs. An object that user code constructs is an ordinary object, however: instances of classes whose instances are tracked in memory (by default, the subclasses of `Symbol` other than roles and predicates) are registered there on construction, where variables without a given domain find them.

Every $f \in \mathcal{F}$ and every characteristic function $p \in \mathcal{P}$, and every other piece of user code that evaluation invokes (property getters and overridden comparison, hashing and iteration methods), is assumed **pure** (no side effects on $\mathcal{A}$ or shared state) and **deterministic** (same inputs always produce the same output), and **terminating** on every input that evaluation passes to it. These conditions are required at all times to ensure correctness and termination of query evaluation. An acyclic call graph over $\mathcal{F} \cup \mathcal{P}$ rules out unbounded mutual recursion between callables, but it does not imply termination, since a single non-recursive callable may loop. These are semantic preconditions (Section 5.2); they are not statically checkable for arbitrary Python callables [10].

2.2 The Knowledge Base

Definition 2.1 (EQL Knowledge Base). An EQL *knowledge base* is a pair $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ where:

- $\mathcal{T}$ (*TBox*) is the Python class hierarchy: the set of class symbols $\mathcal{S}$, their attribute symbols $\mathcal{M}$, and the subsumption ordering $\sqsubseteq$, together with the classes' axioms and the predicates and functions. The property descriptors of KRROOD, which apply property rules when a value is assigned, are abstracted away here: their consequences are already part of the attribute values.
- $\mathcal{A}$ (*ABox*) is a finite set of ground typed object instances. $\mathcal{A}$ contains no disjunctive assertions, no conditional facts, and no evaluable statements.

2.3 Variable Symbols

Let $\mathcal{V} = \mathcal{V}_Q \cup \mathcal{V}_I$ be the set of *variable symbols*. These are the logical symbols of EQL, ranging over domains of constants drawn from $\mathcal{C}$. EQL provides two types of variables:

Query variables $\mathcal{V}_Q$: each carries a class annotation $S \in \mathcal{S}$ and a finite *domain* D , written `variable(S, domain=D)`; only the elements of D that are instances of S are admissible. If D is omitted, it is the set of instances of S in memory, for classes whose instances are tracked; for other classes a domain must be given. `variable_from(D)` creates a variable from a domain

alone. A domain may also depend on other variables: `flat_variable(s.takes_course)` ranges over the elements of the attribute `takes_course` of the value of `s`. We write $\llbracket D_x \rrbracket_\theta$ for the domain of x under an assignment θ of the variables it depends on. Query variables are used in query bodies and heads.

Inference variables $\mathcal{V}_I$: each carries a class annotation $S \in \mathcal{S}$ and constructor keyword arguments $\bar{k}$ mapping field names to body variables. An inference variable carries no pre-specified domain; its domain is populated at rule invocation time:

$$D_{x_I} = \{ H(\bar{k}(\bar{a})) \mid \bar{a} \in \text{ans}(Q_{\text{body}}, \mathcal{K}) \}$$

In EQL, inference variables are constructed via `inference(H)(**kwargs)`. For a plain class H , a derived instance is identified, like a Skolem term $f_H(\bar{k})$, by its class and its argument binding, in which objects are compared by identity and literal values by value and type: evaluating the rule again, or another rule with the same class and binding, returns the same object as long as that object is still referenced. This reuse can be switched off with `inference(H, reuse_inferred_objects=False)`; predicates and symbolic functions are called on every answer.

3 Syntax

This section defines the *abstract syntax* of EQL: the grammar of well-formed terms, atoms, formulas, queries, and rules, independently of their Python surface representation. The Python API constitutes a concrete realisation of this abstract language; where relevant, correspondences to specific API constructs are noted as glosses (e.g., $\wedge$ is expressed in EQL via `and_()` or multiple arguments to `.where()`).

3.1 Terms

Terms are defined inductively over Σ :

$$t ::= c \mid x \mid t.m \mid t.m[i] \mid t.m(\bar{t}) \mid f(t_1, \dots, t_n)$$

where:

- $c \in \mathcal{C}$ is a constant (ground object), also called a *Literal* in EQL; e.g., a specific `Robot` instance.
- $x \in \mathcal{V}$ is a variable with a class and a domain, e.g. r in `variable(Robot, domain=robots)`.
- $t.m$ is a field access on term t , returning the value of field m ; e.g., `r.battery`. Access chains such as $t.m.b$ are admitted, since $t.m$ is itself a term; e.g., `r.tasks[0].completed`.
- $t.m[i]$ is an index access retrieving the element at position i (or with key i for a dictionary) of the indexable field m of t ; e.g., `r.tasks[0]`.
- $t.m(\bar{t})$ is a method call invoking method m on t with arguments $\bar{t}$; when t or any element of $\bar{t}$ is an unground variable, the call is deferred; e.g., `r.name.startswith("R")`.
- $f(t_1, \dots, t_n)$ is an explicit symbolic function application; when any t_i is an unground variable, evaluation is deferred; e.g., `distance(r.position, base)`.

3.2 Atomic Formulas

An *atom* has one of the following forms:

$$\alpha ::= p(t_1, \dots, t_n) \mid t_1 \text{ op } t_2 \mid [t]_{\mathbb{B}}$$

- $p(t_1, \dots, t_n)$ is a *predicate atom*: it denotes the instance of predicate class p constructed with terms $t_1, \dots, t_n$ as arguments. Its truth value is determined at evaluation time by Boolean-casting the ground instance (Section 5.7); e.g., `HasIncompleteTask(r)`.
- $t_1 \text{ op } t_2$ where $\text{op} \in \{=, \neq, <, \leq, >, \geq, \in\}$ is a *comparison atom* over two terms; e.g., `r.battery > 50`.
- $[t]_{\mathbb{B}}$ is a *Boolean term atom*: any term t that evaluates to a Boolean value may appear as an atom by Boolean-casting its value. This covers symbolic function calls ($f(t_1, \dots, t_n)$ where f returns `bool`), method calls ($t.m(\bar{u})$ returning `bool`), and Boolean field accesses ($t.m$ of Boolean type); e.g., `r.name.startswith("R")` or `is_valid(r)`.

Remark 3.1 (Built-in predicates). EQL provides built-in predicate implementations for common TBox queries, including `HasType`, which wraps `isinstance`, and `IsSubclass`, which wraps `issubclass`. These are members of $\mathcal{P}$ like any user-defined predicate and receive no special syntactic treatment.

3.3 Formulas (Query Bodies)

A *body formula* ϕ is built from atoms by:

$$\phi ::= \alpha \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \neg \phi \mid \exists x. \phi \mid \forall x. \phi$$

- Conjunctions ($\wedge$) are expressed via multiple arguments to `.where()` or `and_()`.
- Disjunctions ($\vee$) are expressed via `or_()`.
- Negation ($\neg$) is interpreted as Negation as Failure (Section 5.5), expressed via `not_()`.
- Existential quantification is written `exists(x,  $\phi$ )`. It also arises implicitly: a body variable that is not selected in the head is existentially quantified ($\bar{y}$ in Section 3.4).
- Universal quantification is written `for_all(x,  $\phi$ )` and is evaluated over the finite domain of x .
- The domain of a quantified variable may depend on outer variables, for example the courses of a student s , written `flat_variable(s.takes_course)`. Such a quantifier is evaluated separately for each binding of the outer variables. An outer variable stays free in the quantified formula only if it also occurs outside it; a variable that occurs only inside the quantified formula, including in the quantified variable's domain, is quantified together with it.

3.4 Queries

A *query* is:

$$Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$$

where $\bar{x}$ are the head (output) variables, $\bar{y}$ are body-only existentially quantified variables, and ϕ is a body formula. It is a *conjunctive query* (CQ) if ϕ uses only atoms, conjunction and existential quantification.

A *union of conjunctive queries* (UCQ) is a finite disjunction of CQ bodies sharing the same head variables, expressed in EQL via `or_()` in a single `.where()` call:

$$Q(\bar{x}) \leftarrow \phi_1(\bar{x}) \vee \dots \vee \phi_k(\bar{x})$$

The same-head requirement ensures the result is a well-typed relation: all disjuncts contribute tuples of the same arity and type. In EQL this constraint is enforced architecturally: `or_()` operates inside the body formula ϕ of a single `entity()` or `set_of()` call, so the head variables are shared by construction across all disjuncts.

3.5 Result Quantifiers

A *result quantifier* wraps a query and enforces a cardinality constraint on its answer set, producing a *quantified query* that is itself a query. The constraint is checked incrementally while answers are produced. When it is violated, evaluation raises a `QuantificationNotSatisfiedError`; answers produced before the violation was detected have already been delivered (for example, `.first()` of a `the` query with two answers returns the first).

A *cardinality constraint* $C : \mathbb{N} \rightarrow \{\top, \perp\}$ is one of:

$$C ::= \text{AtLeast}(n) \mid \text{AtMost}(n) \mid \text{Exactly}(n) \mid \text{Range}(\text{AtLeast}(m), \text{AtMost}(n))$$

with the obvious interpretations over the cardinality $|\text{ans}(Q, \mathcal{K})|$.

A *quantified query* pairs a query Q with a constraint C :

$$\mathcal{Q}(Q, C) = \begin{cases} \text{ans}(Q, \mathcal{K}) & \text{if } C(|\text{ans}(Q, \mathcal{K})|) \\ \text{error} & \text{otherwise} \end{cases}$$

EQL provides two quantifier constructors. `an` is the general form: without a constraint argument it applies the trivially satisfied constraint $C(k) = \top$ for all k , acting as a declarative annotation that zero or more results are expected. With a constraint argument it enforces the given C . `the` takes no constraint argument and asserts that exactly one result exists (the constraint `ExactlyOne`); it raises `NoSolutionFound` if there is none and `MultipleSolutionFound` if there are more. It corresponds to Russell’s definite description $\iota x. \phi(x)$ [11], which denotes the unique individual satisfying ϕ and is undefined if no such individual exists or more than one does.

3.6 Rules

A *rule* pairs a query body with an inference variable that constructs new instances from the body’s answer set:

$$x_I \leftarrow \phi(\bar{x}, \bar{y}) \quad x_I \in \mathcal{V}_I, \quad \bar{x} \in \mathcal{V}_Q$$

Rules are not stored in $\mathcal{K}$. On user invocation, the engine evaluates ϕ over the current $\mathcal{A}$, uses the answer set to populate D_{x_I} , and returns the resulting instances, identified like Skolem terms (Section 2.3). The derived instances are ordinary objects (Section 2). There is no fixpoint iteration; each invocation is a single query evaluation.

4 Illustrative Example

The following example is given to ground the abstract definitions in Sections 3 and 5. The domain in this example consists of two classes:

```

@dataclass
class Task:
    name: str
    completed: bool

@dataclass
class Robot:
    name: str
    battery: int
    tasks: List[Task]

```

The fact base $\mathcal{A}$ contains three `Robot` instances and two `Task` instances. The class hierarchy $\mathcal{T}$ contains `Robot` and `Task` as class symbols.

Example query. Retrieve all robots whose battery exceeds 50 and whose first task is not yet completed:

```

r = variable(Robot, domain=robots)
results = an(entity(r).where(r.battery > 50, not_(r.tasks[0].completed))
            ).tolist()

```

In abstract syntax, this is the CQ $Q(r) \leftarrow \phi(r)$ where:

$$\phi(r) = (r.battery > 50) \wedge \neg(r.tasks[0].completed)$$

Here $r \in \mathcal{V}_Q$, $r.battery$ is a field access term (Section 3), $r.tasks[0].completed$ is an index access chained with a field access, $r.battery > 50$ is a comparison atom, and $\neg$ is NAF negation (Section 5.5).

Example rule. Derive `HighLoadRobot` instances for robots whose battery is below 30 and that have at least one incomplete task:

```

@dataclass
class HighLoadRobot:
    robot: Robot

@symbolic_function
def has_incomplete_task(robot):
    return any(not t.completed for t in robot.tasks)

r = variable(Robot, domain=robots)
high_load = inference(HighLoadRobot)(robot=r)
rule = an(entity(high_load).where(r.battery < 30, has_incomplete_task(r)
                                ))

```

In abstract syntax this is the rule $x_I \leftarrow \phi(r)$ where:

$$\phi(r) = (r.battery < 30) \wedge [has_incomplete_task(r)]_{\mathbb{B}}$$

Here $r \in \mathcal{V}_Q$ is a query variable, $x_I \in \mathcal{V}_I$ is an inference variable of class `HighLoadRobot` with argument mapping `robot` $\mapsto r$ (Section 2.3), and $[has_incomplete_task(r)]_{\mathbb{B}}$ is a Boolean term atom over a symbolic function (Section 3.2). For each binding of r satisfying ϕ , the rule derives the instance `HighLoadRobot(robot=r)`; these instances form the rule's answer set D_{x_I} , and evaluating the rule again returns the same instances.

5 Semantics

5.1 Domain Closure and Unique Names

Definition 5.1 (Domain Closure Assumption). Let $\tau : \mathcal{C} \rightarrow \mathcal{S}$ be a *typing function* that assigns each constant its most specific class. The *domain* of class $S \in \mathcal{S}$ is:

$$\text{dom}(S) = \{c \in \mathcal{C} \mid \tau(c) \sqsubseteq S\}$$

where $\sqsubseteq$ is the subsumption ordering of $\mathcal{T}$. No individuals exist beyond those in $\mathcal{A}$.

In EQL, τ is realised by Python’s class system and domain membership $\tau(c) \sqsubseteq S$ can be checked at runtime using `isinstance(c, S)`.

Every query variable $x \in \mathcal{V}_Q$ ranges over a finite domain, given or taken from the objects in memory. Inference variables $x_I \in \mathcal{V}_I$ are domain-free at construction; their domain is populated from the rule body’s answer set at invocation time.

Objects and individuals. Objects and individuals do not correspond one to one in KR-ROOD, because of the Role pattern: a role object refers to the object playing it, its *role taker*. We write $\iota(o)$ for the individual that object o denotes: a role object denotes the individual of its role taker, an object created for an ontology individual denotes the individual named by its IRI, and every other object denotes an individual of its own; on data values, ι is the identity. The *unique name assumption* reads: two objects denote the same individual only if they carry the same IRI or one is a role, possibly of a role, of the other. Together with domain closure, this fixes a single finite interpretation $\mathcal{I}_{\mathcal{A}}$ of $\mathcal{K}$ [2]: a class denotes the individuals of its instances, an attribute relates the individual of every object to the individuals of its values, a predicate denotes the tuples for which its code returns true, and a symbolic function the function its code computes. The extensions of predicates and functions are not stored but computed when evaluation needs them.

5.2 Evaluation Semantics

EQL evaluates queries by nested iteration over the variables’ domains, a left-deep nested-loop join. Domains may be any iterable; their finiteness is a precondition. With finite domains, evaluation makes finitely many invocations of user code (Section 6), so it terminates whenever every invoked piece of user code terminates (Section 2).

Given $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$ and $\mathcal{K} = (\mathcal{T}, \mathcal{A})$, evaluation proceeds left to right through the expression tree of ϕ :

1. **Lazy variable binding.** Variables are not enumerated upfront. Each variable $x_i \in \mathcal{V}_Q$ is bound when its domain D_i is first encountered in the left-to-right traversal of ϕ . The engine iterates over D_i in a nested loop, passing each candidate binding forward to the next subexpression.
2. **Short-circuit filtering.** Under $\wedge$: if a subexpression fails for the current binding θ , the remaining conjuncts are not evaluated for that θ , and the next candidate is tried. Under $\vee$: if the left disjunct succeeds, the right is skipped. No enumeration is performed beyond what the expression requires.
3. **Head projection.** After ϕ is satisfied, any head variable $x_i \in \bar{x}$ not constrained by ϕ is joined with the current bindings via a Cartesian product over D_i to produce complete result tuples.
4. **Result collection.** All bindings θ for which ϕ evaluates to true are collected; projecting onto $\bar{x}$ yields the answer set.

5.3 Answer Set

Definition 5.2 (Answer Set). The *answer set* of query $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$ with respect to $\mathcal{K}$ is:

$$\text{ans}(Q, \mathcal{K}) = \{ \theta(\bar{x}) \mid \mathcal{I}_{\mathcal{A}} \models \phi\theta \}$$

where θ ranges over the assignments of values to all variables $\bar{x}\bar{y}$ of the head and the body with $\theta(x) \in \llbracket D_x \rrbracket_{\theta}$ for each of them (a domain may depend on other variables, also on body variables, as in `flat_variable(dean.teaches_course)`), $\models$ is first-order satisfaction, and the quantifiers in ϕ range over the variables' domains in the same way. Negation is negation as failure (Section 5.5).

As in SQL, evaluation produces answers with bag semantics, one for every satisfying binding of the body's variables; `distinct` removes duplicates and yields the set above.

5.4 Aggregation Semantics

An *aggregated query* is a semantic transformation applied to the answers of a query, corresponding to the γ operator of relational algebra [5, 2] and to SQL's `GROUP BY` pattern.

Let $\text{Agg} = \{\text{count}, \text{sum}, \text{avg}, \text{max}, \text{min}, \text{mode}, \text{multimode}\}$ be the set of aggregate functions, each mapping a multiset of values to a scalar or set of scalars. Given a query $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$, a grouping key $\bar{g} \subseteq \bar{x}$, and aggregate expressions $f_1(e_1), \dots, f_k(e_k)$ where each e_i is a term over $\bar{x}\bar{y}$ (e.g. `count(c)` for a body variable c), the aggregated query evaluates as:

$$\gamma_{\bar{g}; e_1 \rightarrow f_1, \dots, e_k \rightarrow f_k}(B(Q, \mathcal{K}))$$

where $B(Q, \mathcal{K})$ is the bag of satisfying bindings that evaluation produces. This partitions $B(Q, \mathcal{K})$ by $\bar{g}$ and computes each f_i over the multiset of e_i values within each group. In EQL this is expressed via `set_of(...).grouped_by( $\bar{g}$ )`. Post-aggregation filtering via `.having()` applies a condition after aggregation, equivalent to SQL's `HAVING`; this is distinct from `.where()`, which filters tuples before grouping as part of ϕ . When no grouping key is specified, f is computed over the entire bag $B(Q, \mathcal{K})$.

5.5 Negation as Failure

EQL supports *Negation as Failure* (NAF) [4]. In Definition 5.2, $\neg\phi\theta$ holds if and only if $\phi\theta$ is not satisfied in $\mathcal{I}_{\mathcal{A}}$. Because $\mathcal{I}_{\mathcal{A}}$ is a single interpretation in which every ground atom, computed atoms included, is either true or false, this is Reiter's *Closed World Assumption* (CWA) [6]: for a database of ground atoms under domain closure and unique names, the CWA determines a model that is unique up to isomorphism. Operationally, $\neg\phi$ succeeds in a query body if and only if ϕ fails finitely during evaluation. NAF is expressed in EQL via `not_()`.

NAF is **non-monotonic**: adding new facts to $\mathcal{A}$ may invalidate previously derived negative conclusions. EQL does not provide monotonic or paraconsistent negation.

5.6 Semantics of Terms

Under a ground substitution θ , terms evaluate as follows.

Field access.

$$\llbracket t.m \rrbracket_{\theta} = \text{the value of field } m \text{ on object } \llbracket t \rrbracket_{\theta}$$

Access chains evaluate left-associatively: $\llbracket t.m.b \rrbracket_{\theta} = \llbracket (t.m).b \rrbracket_{\theta}$.

Index access.

$$\llbracket t.m[i] \rrbracket_{\theta} = \llbracket t.m \rrbracket_{\theta}[\llbracket i \rrbracket_{\theta}]$$

i.e., element i of the indexable value of field m on $\llbracket t \rrbracket_{\theta}$.

Method call.

$$\llbracket t.m(\bar{u}) \rrbracket_\theta = \llbracket t.m \rrbracket_\theta(\llbracket \bar{u} \rrbracket_\theta)$$

i.e., method m of $\llbracket t \rrbracket_\theta$ called with ground arguments $\llbracket \bar{u} \rrbracket_\theta$. If t or any element of $\bar{u}$ is unground, the call is deferred as a symbolic function application, returning a new EQL variable.

Symbolic function.

$$\llbracket f(t_1, \dots, t_n) \rrbracket_\theta = f_{\text{impl}}(\llbracket t_1 \rrbracket_\theta, \dots, \llbracket t_n \rrbracket_\theta)$$

where f_{impl} is the underlying callable. If any t_i is unground, evaluation is deferred. Determinism and purity ensure the result is unique and $\mathcal{A}$ is unmodified.

5.7 Semantics of Atoms

Atom evaluation determines whether a ground atom holds under substitution θ . Term evaluation (Section 5.6) is applied first to obtain ground values; truth determination is a separate subsequent step.

Predicate atom. Applying θ to $p(t_1, \dots, t_n)$ yields the ground predicate instance:

$$\llbracket p(t_1, \dots, t_n) \rrbracket_\theta = p(\llbracket t_1 \rrbracket_\theta, \dots, \llbracket t_n \rrbracket_\theta)$$

The atom *holds* if and only if the Boolean cast of this instance returns **True**:

$$\alpha \text{ holds under } \theta \iff \text{bool}(\llbracket p(t_1, \dots, t_n) \rrbracket_\theta) = \text{True}$$

The Boolean cast invokes the characteristic function `__call__` of the instance.

Comparison atom. The atom $t_1 \text{ op } t_2$ holds under θ if and only if:

$$\llbracket t_1 \rrbracket_\theta \text{ op } \llbracket t_2 \rrbracket_\theta$$

Boolean term atom. The atom $[t]_{\mathbb{B}}$ holds under θ if and only if:

$$\text{bool}(\llbracket t \rrbracket_\theta) = \text{True}$$

This applies to any term evaluating to a Boolean value: symbolic function calls, method calls, and Boolean field accesses are all admitted as atoms via this form.

6 Computational Complexity

Data complexity fixes the query and measures cost as $|\mathcal{A}|$ grows—the relevant measure for a deployed system. *Combined complexity* lets both the query and $\mathcal{A}$ vary [13]. Table 1 summarises the results; each row adds one feature to the row above, so the bounds can only grow from one row to the next.

The entries follow from established results:

- **CQ and UCQ.** CQ evaluation is NP-complete in combined complexity [3]; UCQ shares the same bounds, and both are in AC^0 in data complexity [2].
- **+ NAF and $\forall$.** With negation and universal quantification over finite domains, query bodies are first-order formulas evaluated under the active-domain semantics. Their combined complexity is PSPACE-complete [13]: nested alternations of $\forall$ and $\exists$ encode quantified Boolean formulas, so unfolding $\forall x \in D$ into a finite conjunction does not lower the bound. Their data complexity is in AC^0 [7, 8].

Table 1: Complexity of EQL query evaluation by feature class.

Feature class	Data complexity	Combined complexity
CQ and UCQ	AC^0	NP-complete
+ NAF and $\forall$ (first-order logic, active domain)	AC^0	PSPACE-complete
+ aggregation	TC^0	PSPACE-complete ^a
+ predicates and symbolic functions (as oracles)	FP^O	PSPACE-complete ^b

^a For **count**, **sum**, **min**, **max** and **avg** over numbers of polynomial size.

^b If every callable runs in polynomial time and every value computed has size polynomial in the size of $\mathcal{A}$ and the query.

- **+ aggregation.** Counting aggregates can express majority, which is not in AC^0 ; first-order logic with aggregates has data complexity in TC^0 [9, 12].
- **+ predicates and symbolic functions.** We treat callables as oracles. A query ϕ with k variables, including quantified ones, whose domains have size at most d , invokes callables at most $|\phi| \cdot d^k$ times. Domains may be collections that the program computes, so we measure cost in $|\mathcal{A}|$ and d . For a fixed query and domains of size polynomial in $|\mathcal{A}|$, this is polynomial, so evaluation is in FP^O , and it runs in polynomial time whenever every callable does. In combined complexity, nesting callables can make values grow exponentially, hence the condition on value sizes in footnote *b*. Termination of arbitrary callables cannot be guaranteed (Section 2).

In practice, most EQL queries involve at least one predicate or symbolic function, so the last row is the operative bound.

References

- [1] F. Baader, D. Calvanese, D. McGuinness, D. Nardi, and P. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications*. Cambridge University Press, 2003.
- [2] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison-Wesley, 1995.
- [3] A. K. Chandra and P. M. Merlin. Optimal implementation of conjunctive queries in relational databases. In *Proc. 9th Annual ACM Symposium on Theory of Computing (STOC)*, pages 77–90. ACM, 1977.
- [4] K. L. Clark. Negation as failure. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 293–322. Plenum Press, 1978.
- [5] A. C. Klug. Equivalence of relational algebra and relational calculus query languages having aggregate functions. *Journal of the ACM*, 29(3):699–717, 1982.
- [6] R. Reiter. On closed world data bases. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 55–76. Plenum Press, 1978.
- [7] N. Immerman. Languages that capture complexity classes. *SIAM Journal on Computing*, 16(4):760–778, 1987.
- [8] D. A. Mix Barrington, N. Immerman, and H. Straubing. On uniformity within NC^1 . *Journal of Computer and System Sciences*, 41(3):274–306, 1990.
- [9] L. Hella, L. Libkin, J. Nurmonen, and L. Wong. Logics with aggregate operators. *Journal of the ACM*, 48(4):880–907, 2001.
- [10] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.
- [11] B. Russell. On denoting. *Mind*, 14(56):479–493, 1905.
- [12] L. Libkin. Expressive power of SQL. *Theoretical Computer Science*, 296(3):379–404, 2003.
- [13] M. Y. Vardi. The complexity of relational query languages. In *Proc. 14th Annual ACM Symposium on Theory of Computing (STOC)*, pages 137–146. ACM, 1982.